Project report for Autonomous Robots

Autonomous Navigation of a RosBot in different Mazes in Webots

Farrukh Rasool | Muhammad Aaliyan | Olatunji Afolabi[1]

[1] *OTH Amberg-Weiden, Study Programme "Master of Artificial Intelligence for Industrial Applications"*

Report written on July 9, 2026

Abstract

This project builds a fully autonomous controller for a Husarion RosBot inside the Webots simulator. The robot has to explore an unknown maze using only its own onboard sensors (no Webots Supervisor access, no hand-placed waypoints), build a map of its surroundings as it goes, find a blue pillar and drive to it, and then find and drive to a yellow pillar, all while never touching the green forbidden ground and never crashing into ordinary walls or the floating walls that some mazes contain. The solution combines odometry, a particle-filter SLAM system, an occupancy-grid map, colour-based pillar detection, frontier exploration, A* global path planning, and a Dynamic Window Approach (DWA) local controller, all wrapped in a state machine that sequences "find blue, then find yellow." The same controller code, with no maze-specific tuning, was tested across five different maze worlds and consistently reached both pillars in the correct order. [a][b]

Keywords: *Autonomous Navigation, Mobile Robot, Webots Simulation, Husarion RosBot, SLAM (FastSLAM 2.0), Occupancy Grid Mapping, Frontier-Based Exploration, A* Path Planning, Dynamic Window Approach (DWA), Sensor Fusion, Obstacle Avoidance, Semantic Perception*

[a] "Source code and project directory/history are available at: https://github.com/farrukhrasool01/Autonomous-Robot"
[b] "All the successful simulation run videos discussed in the Results section are available at: https://www.youtube.com/channel/UCJZKE7LBUJYjytxLP-tx9zA"

1. Introduction

The assignment is based on the Husarion RosBot, a four-wheeled differential-drive robot equipped with wheel encoders, an IMU, four short-range distance sensors, an RGB camera, a depth camera, and a 2D laser scanner. It is placed inside a Webots maze that it has never seen before and has to reach a blue pillar first and then a yellow pillar, while the simulation clock keeps running the whole time - so the faster the robot finishes, the better. The maze also contains "forbidden" green ground that must never be driven over, narrow and very narrow passages that the robot must be able to squeeze through (or recognise it cannot), and floating walls that hang above the floor and are invisible to a normal 2D laser but would still collide with the robot's body.

The two hard rules that shape almost every design decision in this project are: (1) the Webots Supervisor API is completely off-limits, meaning the controller can never simply ask the simulator "where am I" or "where is the blue pillar" - everything must come from the robot's own sensors, and (2) the solution must not be hand-fitted to one maze. Because of this, the whole system had to be built the way a real robot would be built: sense, localise, map, detect, decide, and only then move.

2. Problem Statement and Requirements

The task, as given, can be summarised as follows: implement navigation for the RosBot in Webots so that, starting from the beginning of the simulation, the robot first reaches the blue pillar and then the yellow pillar in the shortest possible simulation time (the timer stops the moment the yellow pillar is reached). The robot must never drive over the green ground. The environment can contain narrow passages, passages so narrow they cannot be passed at all, and floating walls that sit above the ground and are not always visible to every sensor. The robot can be assumed never to leave the bounded maze area. Using the Webots Supervisor is strictly forbidden, and neither the robot model nor the world file may be modified. The solution has to work generally, not just on one specific maze layout, since it is tested on five different mazes.

3. System Architecture

The controller is split into many small, single-purpose files instead of one large script, so each concern (sensing, mapping, planning, moving) can be understood, tuned, and tested on its own.

- `devices.py` creates the Webots `Robot` object and enables every sensor and motor once, at import time. Every other file reads hardware handles from here instead of touching Webots directly.
- `config.py` holds every tunable number in the project (speeds, thresholds, grid size, SLAM parameters, planner weights, etc.) in one place, so behaviour can be tuned without touching logic.
- `kinematics.py` and `motion.py` form the actuation layer: they convert a desired body motion (forward speed + turn rate) into individual wheel speeds and write them to the motors.
- `sensors.py` is the only file that reads raw sensor data (laser, cameras, range sensors, encoders, IMU) and turns it into clean, reusable numbers - nothing here decides *what to do*, only *what is out there*.
- `localization.py` turns wheel-encoder and IMU readings into a running estimate of the robot's pose (position + heading), and feeds that motion information into the SLAM system.
- `slam.py` and `pose_graph.py` implement the probabilistic localisation-and-mapping backend (FastSLAM 2.0 with loop closure), which produces the trustworthy pose used by everything downstream.
- `mapping.py` owns the shared occupancy grid - the actual map of free space, walls, forbidden green ground, floating walls, and the two pillars once they are confirmed.
- `perception.py` turns a camera sighting (bearing + distance) into a world-frame position and remembers the last known position of each pillar, even after it leaves the camera's view.
- `reactive.py` and `safety.py` form a lightweight, always-on safety net that overrides *any* commanded motion if it would cause a collision or drive onto green ground.
- `planning.py` computes a global route between two map cells using A* search.
- `following.py` drives that route locally and safely using the DWA algorithm.
- `exploration.py` implements frontier-based exploration so the robot can autonomously discover the unmapped parts of the maze.
- `mission.py` is the top-level "blue then yellow" state machine that ties exploration, perception, planning, and following together into the actual graded task.
- `autonomous.py` is a simpler, purely reactive alternative (wall-following + seek) kept as a fallback/teleop-adjacent mode; it does not use the map or planner at all.
- `visualizer.py` and `sensor_debug.py` are debugging aids (a live map window and console printouts) that are entirely in-

dependent of the navigation logic - removing them would not change robot behaviour.
- `my_controller.py` is the single entry point Webots actually runs. It owns the main control loop and the keyboard, and simply calls into the modules above depending on which mode is active.

The dependency flow is roughly layered: `devices` → `sensors` → `localization/slam` → `mapping` → `planning/exploration/perception` → `following/reactive/safety` → `mission/autonomous` → `my_controller`, with `config.py` read by nearly everything above it.

4. Logical Approach

For each sub-problem the project faced, the guiding idea was:

- **Moving the robot safely** - treat it as a differential-drive vehicle, convert desired motion into wheel speeds, and never trust a single planner blindly: always pass the final command through a hard safety check first.
- **Knowing where the robot is** - wheel encoders drift and slip, so fuse them with the IMU's heading and correct the estimate over time with a proper SLAM algorithm instead of relying on raw odometry.
- **Building a map without a Supervisor** - accumulate evidence from the laser scanner into a probability-based occupancy grid over time, so uncertain areas stay "unknown" until enough evidence accumulates.
- **Avoiding green ground and floating walls** - since neither is a normal laser-visible wall, use the cameras: segment green pixels and project them onto the floor, and separately look at the *upper* part of the depth image (where a laser beam would miss a floating wall) and project that into the map as an obstacle.
- **Finding the pillars** - segment blue/yellow pixels by colour, estimate their direction and distance from the camera, and remember the last known world position so the robot does not "forget" a pillar the moment it looks away.
- **Exploring an unknown maze** - repeatedly look for the boundary between known-free and unknown space ("frontiers"), and drive toward the most useful one.
- **Getting from A to B efficiently and safely** - plan a coarse global route with A* on the map, then let a local controller (DWA) handle the moment-to-moment steering and obstacle avoidance, because the map is never perfectly up to date.
- **Doing "blue, then yellow" in minimal time** - instead of exploring the whole maze blindly, bias exploration toward a pillar as soon as it is glimpsed, and only commit to a precise final approach once close enough for that sighting to be trustworthy.

5. Techniques Used

5.1. Kinematics and motion control

The RosBot is modelled as a differential-drive robot: the two left wheels share one speed, the two right wheels share another. `kinematics.py` converts a desired body twist (linear speed `v`, angular speed `omega`) into the two wheel angular speeds using the standard differential-drive equations, using the robot's real wheel radius (`WHEEL_RADIUS_M = 0.043` m) and wheel track (`WHEEL_TRACK_M = 0.18` m). Before any speed is sent to the motors, it is scaled down (keeping the same `v`/`omega` ratio) if it would ask a wheel to spin faster than `MAX_WHEEL_SPEED_RAD_S = 36` rad/s, so the robot never simply saturates and drives an unintended arc.

5.2. Localization: fused odometry

Position is integrated every simulation step from the average of the front and rear wheel encoders on each side (`localization.py`). Heading, however, comes directly from the IMU's inertial unit rather than from the wheels, because wheel-only heading drifts badly under skid-steering. A slip check compares what the encoders *claim* (a turn) against what the gyroscope actually *measures*: if the wheels say the robot is rotating but the gyro disagrees, the wheels are assumed to be slipping in place (e.g. pinned against a wall), and the phantom motion is discarded (`SLIP_ENC_ROT_RAD`, `SLIP_GYRO_RATE_RADS`) so it cannot corrupt the map.

5.3. SLAM: FastSLAM 2.0 particle filter with loop closure

Odometry alone is not accurate enough to build a clean, consistent map over a whole maze, so the project implements a particle-filter SLAM (`slam.py`), the same family of algorithm used in classic mobile-robotics coursework (FastSLAM). Thirty particles (`SLAM_NUM_PARTICLES`) each carry their own candidate pose *and* their own occupancy map. Every simulation step, each particle's pose is nudged forward using the odometry motion, with a small amount of random noise added (`SLAM_ALPHA1`–`SLAM_ALPHA4`) to represent motion uncertainty. Roughly ten times a second, a background thread takes the current laser scan and, for each particle, first *refines* that particle's pose with a small local search against its own map (checking small position and angle offsets, `SLAM_REFINE_RADIUS_PX` / `SLAM_REFINE_WINDOW_DEG` / `SLAM_REFINE_STEP_DEG`) - this is the "improved proposal" step that gives FastSLAM **2.0** its name and keeps each particle's own map sharp. The residual matching error becomes that particle's importance weight (scaled by `SLAM_LIKELIHOOD_SIGMA_M`), and particles are resampled whenever the effective sample size drops below half the population (`SLAM_RESAMPLE_NEFF_RATIO`), so unlikely particles are replaced by copies of likely ones. The best/representative particle's map is then published as the "official" map everyone else reads.

Running this on a background thread (rather than inside the main control loop) is what keeps the robot able to move every simulation step even though the particle-filter update is comparatively expensive; a lock protects the small amount of shared state so the main loop and the mapping thread never corrupt each other's data. To save CPU, a scan is only folded in once the robot has actually moved a minimum distance or turned a minimum angle since the last update (`SLAM_OBSERVE_MIN_TRANS_M`, `SLAM_OBSERVE_MIN_ROT_RAD`), instead of on a fixed timer.

On top of the particle filter, `pose_graph.py` implements loop closure: as the robot moves, "keyframes" (pose + scan snapshots) are recorded whenever it has moved or turned enough (`KEYFRAME_DIST_THRESHOLD_M`, `KEYFRAME_ANGLE_THRESHOLD_DEG`). When a new keyframe is nearby (in raw position) to an older one, a small scan-matching search (`LOOP_CLOSURE_SEARCH_RADIUS_M`, `LOOP_CLOSURE_SEARCH_WINDOW_M/DEG`) checks whether the two scans actually line up. If they match well enough (`LOOP_CLOSURE_SCORE_THRESHOLD`), the whole chain of keyframe poses is re-optimised with a least-squares solver, and the map is rebuilt from scratch using the corrected poses - this is what fixes the accumulated drift from a long loop through the maze back to somewhere already visited. Cooldowns and attempt-throttling (`LOOP_CLOSURE_COOLDOWN_KEYFRAMES`, `LOOP_CLOSURE_ATTEMPT_INTERVAL`) stop this relatively expensive check from being attempted every single keyframe as the map grows.

5.4. Occupancy mapping

`mapping.py` keeps a 300 × 300 cell grid covering a 10 m × 10 m area (about 3.3 cm per cell, `MAP_SIZE`, `MAP_RES_M`), using the classic **log-odds occupancy grid** technique. For every laser scan, a Bresenham line is drawn from the robot's cell to each measured point: cells the beam passes through get a small "probably free" nudge (`LOGODDS_FREE`), and the cell the beam actually hits gets a "probably occupied" nudge (`LOGODDS_OCC`). Once a cell's accumulated evidence is strong enough in either direction it is "locked" (`LOGODDS_LOCK`) so

a confirmed wall can never be eroded by a later stray reading, and the grid is only converted into discrete Free/Occupied/Unknown labels by comparing the resulting probability against thresholds (`P_OCC`, `P_FREE`).

5.5. Forbidden green ground

Green ground cannot be seen by the laser (it is flush with the floor, not a wall), so it is detected purely with the RGB camera: pixels are segmented in HSV colour space (`GREEN_HSV_LOWER/UPPER`), and only the lower half of the image is trusted, since that is where the floor is. Using the camera's known mounting height and field of view, each surviving green pixel is projected onto the floor plane with simple trigonometry (an inverse pinhole-camera model), giving a set of real-world floor points that are stamped into the map as permanently forbidden `CELL_GREEN` cells (`GREEN_CAM_HEIGHT_M`, `GREEN_MAX_PROJ_DIST`). This means the path planner and local controller automatically treat green ground exactly like a wall. As a second line of defence, a fast reactive check (`reactive.py`/`safety.py`) independently stops or steers the robot away the instant green is seen close up (`GREEN_STOP_DIST`, `GREEN_CAUTION_DIST`), so the robot never depends on the map alone for this specific rule.

5.6. Floating walls

Floating walls sit above the floor and, depending on their height, may pass entirely above the laser's single scanning plane - meaning the laser can report a corridor as completely clear even though a wall panel is hanging in front of the robot. To catch this, the *upper* band of the depth camera image is scanned instead: every valid depth pixel in that band is converted to a body-frame 3D point using the same pinhole-camera geometry as the green-ground projection, but this time the point's actual height above the floor is also recovered from the ray geometry. Only points at or below the robot's own body height (`OVERHEAD_ROBOT_CLEARANCE_M`) are kept - a floating wall high enough for the robot to drive underneath is deliberately *not* marked. Surviving points are stamped as a special "closed" obstacle cell (`CELL_CLOSED`) that both the planner and the local follower treat as a hard wall. A safety cross-check demotes a `CELL_CLOSED` cell back to an ordinary wall if the laser *also* confirms occupancy there, since a genuine floating wall is invisible to a clean laser beam passing underneath it - so agreement between the two sensors means it was never really "floating" in the first place.

5.7. Reactive safety net

Independent of any planner, `safety.py` re-checks every commanded motion before it reaches the motors: if the front laser or front short-range sensors read below a hard stop distance, the command is overridden into a decisive turn toward whichever side has more clearance; if that turn does not clear the blockage within a timeout, the robot reverses out and tries the other side instead (useful for dead ends and tight corners). Reversing is itself blocked if the rear sensors are too close to something. This net is applied on top of *every* mode (teleop, reactive wall-following, mission driving), so a bug or gap anywhere upstream cannot make the robot ram a wall.

5.8. Detecting the blue and yellow pillars

The same HSV-segmentation idea used for green is reused for blue and yellow (`BLUE_HSV_LOWER/UPPER`, `YELLOW_HSV_LOWER/UPPER`). Once a colour's pixel count is large enough to count as "seen" (`TARGET_PIXEL_RATIO`), its bearing is taken from the horizontal position of the colour blob's centroid, converted into an angle using the camera's field of view, and its distance is estimated from the far edge of the depth reading under that blob combined with the pillar's known physical height, using Pythagoras to remove the vertical component (`COLUMN_HEIGHT_CM`). This distance estimate is good for rough navigation but becomes unreliable up close (it saturates) and badly underestimates far pillars, so it is explicitly never trusted to decide "we have arrived" - see the mission logic below. `perception.py` converts a bearing+distance sighting into a world-frame coordinate and keeps the last known position of each pillar in memory, so the robot can still head toward a pillar even after turning away from it.

5.9. Frontier-based exploration

To autonomously discover an unknown maze, `exploration.py` repeatedly looks for **frontiers**: free cells that sit directly next to unknown cells, i.e. the boundary of what has been mapped so far. Neighbouring frontier cells are grouped into clusters (BFS flood-fill), tiny clusters are discarded as noise (`FRONTIER_MIN_CLUSTER`), and each remaining cluster is scored by a simple "bigger and closer is better" formula (`size / (distance + FRONTIER_SCORE_BIAS)`). If a target pillar has already been glimpsed, the score is boosted for frontiers that also point roughly toward it (`PILLAR_BIAS_WEIGHT`), so exploration organically steers toward the sighted pillar instead of wandering purely for coverage. If no frontier is usable, the robot falls back to a random nearby already-mapped free cell, and if even that fails, it simply rotates in place to reveal new space rather than freezing - a small blacklist of already-tried frontiers (which is periodically forgotten) stops it from repeatedly retrying a hopeless target.

5.10. Global path planning (A*)

`planning.py` runs an A* search over the occupancy grid to connect two cells with a smooth, obstacle-free route. Before searching, the grid is cleaned of tiny noise blobs, obstacles (including green and floating-wall cells) are "inflated" outward by a safety margin, and the search itself adds an extra cost near any nearby wall (`ASTAR_SAFE_DISTANCE_PX`, `ASTAR_PENALTY_STRENGTH`) so the chosen route hugs the centre of a corridor rather than skimming a wall. If the safety margin is so wide that no route can be found (a very narrow passage), the margin is progressively relaxed (`ASTAR_INFLATION_LEVELS = [4, 3, 2]`) until a route of reasonable length is found. The raw grid-cell path is then smoothed into a nicer curve with a B-spline. Two variants exist: the strict "final goal" planner refuses to route through unmapped space at all (so the robot can never blindly drive into an unknown wall), while a separate frontier planner deliberately allows routing through unknown space, since reaching the edge of the unknown is the entire point of exploration.

5.11. Local path following (DWA)

A plan on a slightly stale map is not enough on its own - the map keeps changing as the robot moves. `following.py` implements the **Dynamic Window Approach**: every control step, a small set of candidate (`v`, `omega`) pairs is simulated forward for about a third of a second (`DWA_ROLLOUT_STEPS`), and each candidate trajectory is scored on how well it faces the current target waypoint, how much closer it gets, how fast it is, and how much clearance it keeps from mapped obstacles (`DWA_HEADING_WEIGHT`, `DWA_DISTANCE_WEIGHT`, `DWA_SPEED_WEIGHT`, `DWA_CLEARANCE_WEIGHT`). Because DWA scores the *centreline* of a trajectory, a route that only grazes a wall could still let the physical robot body clip a corner, so a soft penalty is added whenever a trajectory's endpoint comes closer to a wall than the robot's own half-width (`DWA_ROBOT_CLEAR_PX`, `DWA_CLEAR_PENALTY`). Progress toward the goal is watched over a short window; if the robot is not actually getting closer, a short reverse-and-turn recovery manoeuvre runs automatically, and if that keeps failing the caller is told to replan a fresh route. A "speed governor" additionally throttles forward speed using the *live* laser reading (not the map) whenever the front is close to something not yet mapped, since DWA can otherwise drive confidently at full speed into a wall it simply has not discovered yet.

5.12. Mission sequencing: reaching blue, then yellow

`mission.py` is the state machine that actually satisfies the graded task, cycling through `SEEKING_BLUE` → `SEEKING_YELLOW` → `DONE`. While seeking a colour, the robot explores (biased toward that colour once glimpsed) until it is either confirmed at close range or within a commit distance of its last sighting (`PILLAR_COMMIT_DIST_M`); only then does it switch from "explore toward it" to a precise final approach - planning a route to a small stand-off point just in front of the pillar (`APPROACH_OFFSET_M`) and driving it with DWA, backed up by a direct "seek" behaviour that centres the pillar in the camera view and drives straight at it once it is close and clearly visible. A pillar is only ever formally "reached and marked" on the map once a *reliable* signal confirms it - either the pillar fills a large fraction of the camera frame, or the front laser is at contact range while the pillar is centred in view - deliberately **not** the depth-based distance estimate, since that estimate is known to be unreliable at both very close and very far range. If yellow happens to be spotted while still seeking blue, its position is remembered so the yellow phase can drive straight there afterwards instead of re-exploring from scratch, which is what keeps the overall run efficient.

6. Modes of Robot Available

The controller exposes several distinct operating modes, selected with keyboard keys, so different layers of the stack can be tested independently:

- **Teleop (F / S / A / D / Space)** - direct manual driving forward/back/rotate, with only the basic front-laser stop as a safety check; `T` runs a short two-second motor self-test.
- **G - Legacy reactive autonomous mode** - pure right-hand wall-following combined with the colour-seek behaviour and mission state machine, but with *no* map, no SLAM-driven planning, and no exploration; a simple, fully reactive fallback.
- **E - Frontier exploration only** - runs the full SLAM/mapping/frontier-exploration stack with no pillar mission attached, useful for testing and visualising mapping quality on its own.
- **X - Full mission (the actual graded solution)** - runs the complete blue-then-yellow pipeline: SLAM, mapping, biased exploration, A* planning, DWA following, and the mission state machine together.
- **B / V - Manual goal + plan (developer tool)** - lets the operator mark up to two cells on the current map and ask the A* planner to compute (and save an image of) a route between them, without driving it.
- **Y - Manual path following (developer tool)** - drives the most recently planned route using the DWA follower, replanning automatically if it gets stuck.
- **C - Live map view** - toggles an on-screen OpenCV window showing the live occupancy grid, robot position, current goal, and planned path, for visual debugging.
- **I / L / O / R / M / P - One-shot diagnostics** - print a full sensor snapshot, toggle continuous sensor logging, print the current pose, reset pose/map/mission state, print a map summary (and save it as an image), and print the bearing/distance to the blue and yellow pillars.

7. Results and Observations

Table 1. Maze 1 – Run Timing Data

Run #	Start	Exploration	Total Exploration Time	Mission Start	Mission End	Blue–Yellow
01	3s	1m 17s	1m 14s	1m 17s	2m 5s	48s
02	5.5s	1m 12s	1m 6.5s	1m 12s	1m 50s	38s
03	3.5s	2m 7s	2m 3.5s	2m 7s	2m 29s	22s

Table 2. Maze 2 – Run Timing Data

Run #	Start	Exploration	Total Exploration Time	Mission Start	Mission End	Blue–Yellow
01	48s	2m 15s	1m 45s	2m 15s	2m 56s	41s
02	0	0	0	0	0	0
03	0	0	0	0	0	0

Table 3. Maze 3 – Run Timing Data

Run #	Start	Exploration	Total Exploration Time	Mission Start	Mission End	Blue–Yellow
01	7s	1m 3s	56s	1m 3s	1m 38s	35s
02	4.5s	2m 47.5s	2m 43s	2m 47.5s	3m 19s	31.5s
03	3.5s	2m 7s	2m 3.5s	2m 7s	2m 29s	22s

Table 4. Maze 4 – Run Timing Data

Run #	Start	Exploration	Total Exploration Time	Mission Start	Mission End	Blue–Yellow
01	7s	2m 4s	1m 57s	2m 4s	3m 23s	1m 19s
02	9.7s	1m 46s	1m 36s	1m 46s	3m 5s	1m 19s
03	7.5s	1m 38s	1m 30.5s	1m 38s	3m 39s	2m 1s

Table 5. Maze 5 – Run Timing Data

Run #	Start	Exploration	Total Exploration Time	Mission Start	Mission End	Blue–Yellow
01	4s	45s	41s	45s	1m 05s	20s
02	3.5s	1m 02s	58.5s	1m 02s	1m 24s	22s
03	14.5s	59s	44.5s	59s	1m 20s	21s

The controller was run three times on each of five maze worlds, and the simulation time was logged at several checkpoints: when exploration started, roughly how long exploration itself took, the total time until the mission (blue-then-yellow) phase started, when the mission ended, and how long the final blue-to-yellow leg took on its own.

Across Mazes 1, 3, 4, and 5, total exploration time before both pillars were reliably known ranged from under a minute up to a little over two minutes, and it was clearly the dominant cost in the whole run - the blue-to-yellow leg itself, once both pillars were localised, was usually finished in well under a minute (roughly 20-50 seconds in most runs, though Maze 4 saw one run take close to two minutes, likely due to a larger or more convoluted layout between the two pillars). There is also noticeable run-to-run variance within the *same* maze - for example, Maze 4's exploration time roughly doubles between its fastest and slowest run, and Maze 1 and Maze 3 both show a similar spread. This is expected given the amount of randomness baked into the pipeline: the particle filter's motion noise, the random fallback used when no good frontier is available, and the fact that a slightly different frontier gets picked first can send the robot down a different corridor on two otherwise-identical runs.

Maze 5 and Maze 3 were the two mazes the robot handled most reliably: all three runs on each completed cleanly, with consistent exploration times and a short, clean blue-to-yellow leg, and no notable issues with where the pillars were detected. Maze 1 and Maze 4 were also solved in all three runs, but with a caveat: on some runs the estimated world-frame coordinates of the blue and/or yellow pillar were slightly off from their true position (a small deviance/miscalculation in the camera-depth-based localisation described in the Techniques Used section), which occasionally meant a slightly longer or more roundabout final approach than a perfectly accurate sighting would have produced, even though the pillar was still reached correctly in the end.

Maze 2 was, in the end, **not reliably solved**: out of three attempts, only one run actually completed the full blue-then-yellow mission, and the other two failed to finish. The single successful run recorded the following timings - start at 48 s, exploration running until 2 m 15 s (about 1 m 45 s of total exploration time), mission phase starting at 2 m 15 s and ending at 2 m 56 s, with the blue-to-yellow leg itself taking 41 s. Even this one successful run was not fully clean: the estimated

coordinate of the blue pillar had a measurement error, meaning the position the robot localised and drove to did not exactly match the pillar's true position, even though the robot ultimately still registered it as reached. Maze 2 therefore stands out as the weakest result of the five worlds tested, and is discussed further below.

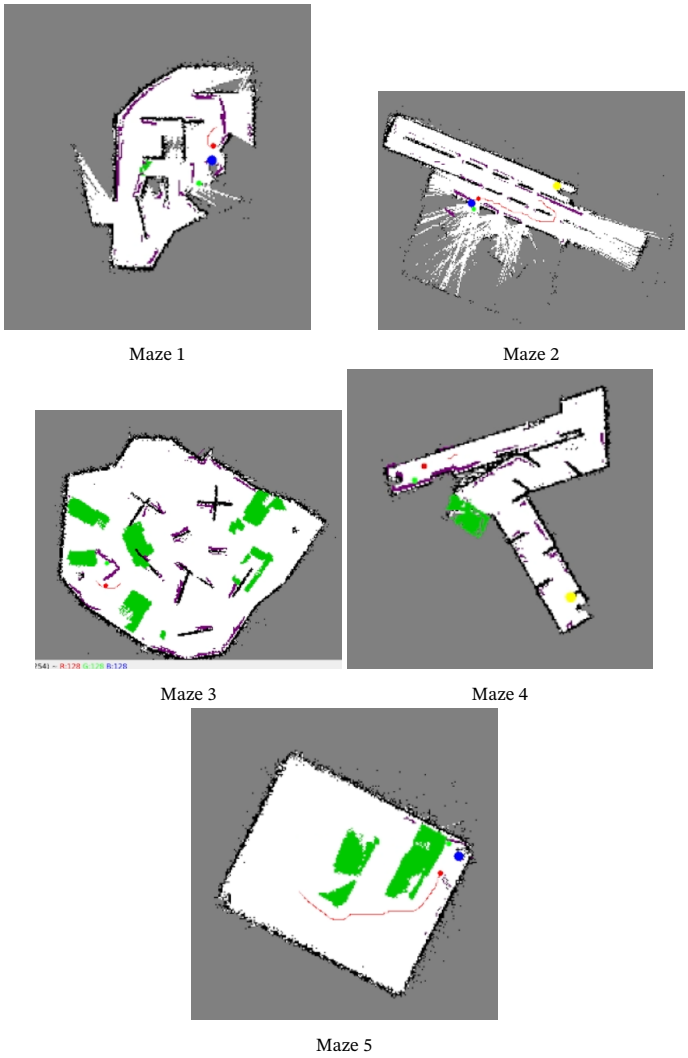

Figure 1. Occupancy grid and A* route captured mid-exploration

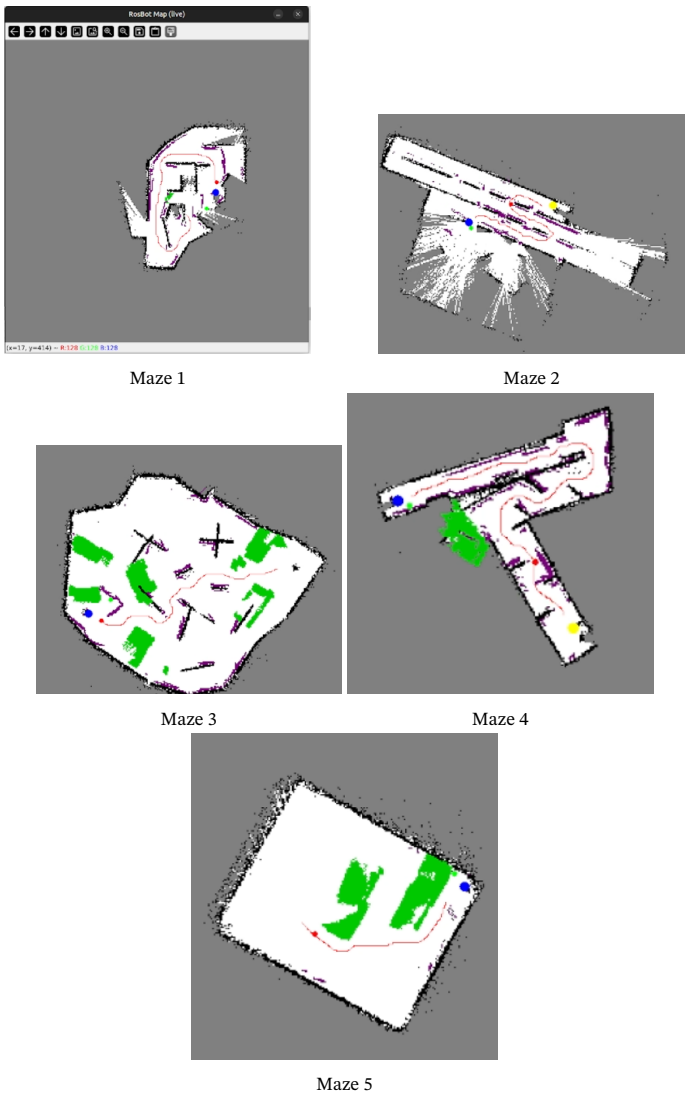

Figure 2. Planning

8. Limitations

The system does not achieve a truly time-optimal route, and this is an expected consequence of the rules rather than a bug: since the Supervisor API cannot be used, the robot genuinely does not know where the pillars are until it has explored enough of the maze to see them, so the majority of the mission time is unavoidably spent mapping rather than driving a shortest path. On top of that, the frontier-selection and particle-filter noise mean run times are not perfectly repeatable - the same maze can take noticeably longer or shorter on different attempts, as seen in the Maze 4 and Maze 1 timing spread.

SLAM pose can still drift somewhat over a long exploration run before a loop closure has a chance to correct it, and in tight or repetitive corridors the correlative scan-matching used for loop closure can occasionally take a while to find a confident match, or not trigger at all if the loop is never revisited closely enough. The floating-wall detector depends on a fixed depth-camera band and clearance-height threshold, so an unusually shaped or unusually mounted floating obstacle at the edges of that assumption could be marked slightly incorrectly (over- or under-blocked). Similarly, the colour-based distance estimate to a pillar is known to be unreliable at long range and saturates at short range, which is why the mission logic deliberately waits for a reliable close-range signal before declaring a pillar "reached" - but this also means the robot sometimes has to get closer than strictly necessary before it is willing to register a pillar, costing a few extra seconds.

This pillar-coordinate inaccuracy was not just a theoretical risk - it was actually observed. In Maze 1 and Maze 4, some runs showed a small deviance between the estimated and true world-frame position of the blue and/or yellow pillar, which came from the same depth-plus-known-height distance estimate discussed above, and occasionally led to a slightly longer or less direct final approach. Maze 2 exposed this same weakness more severely: its single successful run out of three attempts still had a measurable coordinate error on the blue pillar, and the other two attempts failed to complete the mission at all. Maze 2's layout evidently combines conditions (some mixture of tighter passages, less favourable sightlines to the pillars, and/or geometry that is harder for the frontier selector and the SLAM scan-matcher) that this pipeline is not yet robust to, making it the clearest concrete example of the system's generalisation limits rather than a one-off fluke.

In very tight or "too narrow" passages, the planner's escalating safety margin can still fail to find a usable route at the tightest setting, and the local DWA follower can end up in a short cycle of getting stuck, reversing, and replanning before it either finds a way through or the passage is confirmed impassable. Finally, because both the

global planner and the local follower are essentially greedy/local in nature (A* on the current map plus a short-horizon DWA rollout), the overall path is not globally re-optimised as new map information comes in — it reacts well locally, but does not go back and re-plan a shorter route across the whole maze once more of it becomes known.

9. Conclusion

The final controller satisfies every hard constraint of the assignment - no Supervisor access, no maze-specific hardcoding, no modification of the robot or world files - while still autonomously localising itself, mapping an unknown maze, avoiding green ground and floating walls, and reaching the blue pillar before the yellow one. Building it in layers (kinematics → localization/SLAM → mapping → perception/planning/exploration → local following → mission logic), each testable and observable on its own in Webots, made it possible to combine fairly advanced techniques (a FastSLAM 2.0 particle filter with loop closure, log-odds occupancy mapping, A* with clearance-aware costs, and a DWA local controller) into one coherent pipeline. The results across five different, previously unseen mazes show that the approach mostly generalizes rather than being fitted to a single layout: Maze 3 and Maze 5 were handled reliably and cleanly, Maze 1 and Maze 4 were solved consistently but with some pillar-coordinate inaccuracy, and Maze 2 remains an open weak point, having been completed only once in three attempts and even then with a coordinate error on the blue pillar. As expected for an exploration-based strategy, the achieved time is dominated by how much of the maze had to be discovered before the mission could be completed, and individual runs show some natural variance due to the probabilistic nature of the SLAM and exploration components - with Maze 2 showing that this variance can, in a harder layout, tip over into outright failure rather than just a slower run.

A. Repository Layout

Path	Contents
`controllers/my_controller/my_controller.py`	Entry point / control loop
`.../devices.py`	Webots device init (Table **??**)
`.../sensors.py`	Sensor reads, HSV segmentation
`.../kinematics.py`	Differential-drive equations
`.../localization.py`	IMU-fused odometry
`.../mapping.py`	Log-odds occupancy grid
`.../perception.py`	World-frame target projection/memory
`.../exploration.py`	Frontier detection and selection
`.../planning.py`	A* + clearance + spline
`.../following.py`	DWA local path follower
`.../safety.py`	Reactive safety net
`.../mission.py`	Blue→yellow state machine
`.../reactive.py`	Laser sectors, wall-follow twist
`.../config.py`	All tunable constants
`worlds/Maze1-5.wbt`	Maze layouts (generalisation test set)

Table 6. Repository layout referenced throughout this report